Phase 3 developer handoff: runtime foundation

Scope: harden the existing runtime pipeline; do not add a second scanner, receiver or model gateway. Phase 4 evidence entities, digital twin and guided discovery remain gated.

Architecture and ownership

```
SessionInput -> prepare() -> completed tenant/application baseline + Artifact SHA
                          -> build_plan() -> immutable settings JSON
Java agent -> authenticated OTLP route -> bounded decode + normalization
           -> session lock + reauthentication -> resource binding -> deduplication
           -> RuntimeEvent (sanitized fields only) + ingestion counters
RuntimeEvent -> graph builder -> conservative class/method mapping -> coverage
             -> existing fusion/intelligence -> versioned report + quality
Stop -> frozen RuntimeSession.result
Retention -> delete expired RuntimeEvent rows, retain completed report
```

File	Responsibility
`backend/app/runtime/schemas.py`	Validated includes/excludes, binding mode and optional service name; additive report metadata
`backend/app/runtime/routes.py`	Existing management/ingestion APIs, plan preview and diagnostics
`backend/app/services/runtime/instrumentation.py`	Deterministic bounded method selection and canonical plan digest
`backend/app/services/runtime/normalization.py`	Bounded OTLP JSON/protobuf/gzip decoding; architectural/resource allowlists; descriptor validation
`backend/app/services/runtime/mapping.py`	Exact/internal-name/recognized Spring proxy mapping; conservative JVM descriptor conversion
`backend/app/services/runtime/session_service.py`	Baseline association, plan persistence, batch binding, deduplication, lifecycle, quality, recovery and retention
`backend/app/services/runtime/graph_builder.py`	Parent reconstruction, resource identity, malformed-parent-cycle rejection and projection
`backend/app/services/runtime/coverage.py`	Five observation denominators and method-mapping diagnostics
`backend/app/main.py`	Recover interrupted report generation before accepting requests; existing periodic retention sweep
`frontend/src/runtime/`	Session setup, plan preview, quality diagnostics and existing runtime views

Persistence and compatibility

No schema migration is needed; Alembic remains at 0004. New session metadata uses existing `settings` JSON. It includes `binding_mode`, `service_name`, `artifact_sha256`, `exclude_packages`, the saved `instrumentation_plan`, and `ingestion` counters. The bearer token stays hashed in its existing column and never enters the plan/report.

New reports use `derivation_version: runtime-foundation-2`. Saved plans use `version: runtime-plan-2`, selected classes/method names, scope, omission counts, byte budget and a canonical SHA-256 computed without its own hash field. Saved plans are reused

during token rotation/config retrieval. Legacy sessions obtain a transient plan only when needed; their stored snapshots are not rewritten. Existing completed reports are returned as stored; response schemas may add default empty quality/legacy version fields.

The API defaults to COMPATIBLE for existing clients. The UI explicitly submits STRICT. Both reject any supplied service/artifact mismatch and malformed resource identity. Compatible mode accepts absent identity but records it as unverified. The whole batch is rejected atomically with 409 if any span violates binding. Only a numeric rejected-span count is retained. Empty batches do not imply a connection.

Strict binding is a consistency check against a sender-declared SHA-256, not remote attestation. The generated `OTEL_RESOURCE_ATTRIBUTES` must describe the actual analyzed deployment artifact. Future collectors spanning multiple services need an explicit association contract; do not silently accept mixed-service batches.

API additions

- `POST /api/v1/applications/{application_id}/runtime-sessions/plan`: same input as creation; returns a plan without creating a session or token. Requires RUNTIME_WRITE under the existing POST permission policy.
- Creation accepts `exclude_packages`, `binding_mode` and optional `service_name` in addition to existing fields. Unknown fields are rejected.
- `GET /api/v1/runtime-sessions/{id}/diagnostics`: current lifecycle, purge/recovery metadata, ingestion counters, dropped count and saved plan. Requires RUNTIME_READ; no tokens.
- Agent configuration adds `plan`, `binding_mode`, generated service name and artifact resource attributes while preserving the existing `instrumentation_plan` list.
- Evidence adds `quality` and `derivation_version`; coverage adds `method_mapping_diagnostics`.

Tenant/application/baseline checks remain in the existing scoped data/session path. OTLP requires a session bearer and reauthenticates after body parsing inside the process lock, preserving rotation behavior. Model and source permissions are unchanged.

Mapping and quality semantics

Descriptors use JVM syntax, such as `(I)V` or `([Ljava/lang/String;)I`. The mapper prefers a valid static descriptor or conservatively converts a fully qualified jQAssistant method signature. It does not guess unqualified object types or generic signatures. A valid event descriptor must match a static signature; an invalid supplied descriptor cannot fall back to a method name. Without a descriptor, only one distinct static owner/name signature receives coverage. No simple-class-name mapping or arbitrary proxy stripping is introduced.

Duplicate trace/span keys are session-scoped. Exact duplicates increment a counter and do not inflate coverage. Conflicting duplicates preserve the first sanitized event and increment duplicate plus conflict counts. Cumulative counters describe ingestion attempts, not unique workload operations. Default cap remains 50,000 unique spans/session; OTLP partial success identifies storage-cap rejections.

Graph resource IDs now include service (or session when unknown), database server/namespace and message system along with prior type/resource/route fields. Distinct known databases no longer collapse into one resource node. Missing fields still mean unknown; these IDs do not prove resource ownership. Parent reconstruction is scoped to session and trace. Cyclic parent links are omitted; legitimate class-level cycles from valid span trees remain supported. Latency percentiles remain class-edge observations, not end-to-end latency or projected-domain percentiles.

Coverage diagnostics count matching outcomes only where a reliable method inventory exists. Runtime coverage is not JaCoCo/line/branch coverage. The real agent fixture still produced 11 ambiguous method observations; this is reported rather than resolved by guessing.

Lifecycle and retention

CREATED -> WAITING_FOR_AGENT -> RUNNING -> PROCESSING -> COMPLETED; report failure -> FAILED. Start is idempotent only while accepting telemetry. Completed sessions cannot restart. Stop can retry a failed report while raw evidence remains. A server

restart marks interrupted PROCESSING sessions FAILED with a reason/timestamp, preserving active sessions and token hashes.

Retention runs once per minute. Active observation lifetimes are bounded by `runtime_raw_trace_retention_hours`; expired active sessions are finalized. A failure finalizing one session does not prevent later sessions from being processed. Completed/failed raw evidence is purged after the retention cutoff; old failed rows lacking an end timestamp use updated time. Already-purged sessions are skipped, so the returned count reflects newly purged sessions. Completed report JSON is unchanged. The implementation remains single-process; do not deploy multiple API writers until distributed locking/coordination exists.

Verification and reproducibility

Backend contracts: `backend/tests/test_runtime_foundation.py`, alongside runtime/security/AI/scanner regressions. Browser workflow: `frontend/e2e/02-runtime.spec.ts`, included in `playwright.security.config.ts`.

Run from the repository root in PowerShell:

```
$env:ARCHINTEL_RUN_SCANNER_TESTS = '1'
$env:ARCHINTEL_RUN_MODEL_TESTS = '1'
$env:ARCHINTEL_RUN_POSTGRES_TESTS = '1'
$phaseTests = '.data/phase3-tests-' + [guid]::NewGuid().ToString('N')
.\.runtime\python\tools\python.exe -m pytest backend/tests -q -p no:cacheprovider --basetemp=$phaseTests
.\.runtime\python\tools\python.exe scripts/validate_phase3_runtime.py
```

The real-Java script requires the pinned sample JAR, local Java/agent, an existing matching static method-inventory snapshot, and free loopback ports 18002/18080. It reads the source workspace, seeds an isolated SQLite database under `.data/phase3-real-java-*`, bootstraps only that isolated installation, runs trusted PetClinic, verifies traffic/coverage, stops/restarts only its own processes and writes a credential-free report to `docs/phase3-runtime-validation.json`. It leaves the isolated directory/logs for inspection. It does not execute uploaded customer applications. Credentials are randomly generated and are not included in the report.

From `frontend`, use `npm.cmd test -- --run`, `npm.cmd run build`, and `npm.cmd exec playwright test -- --config=playwright.security.config.ts`. Browser tests use their own database on port 18001. The runtime browser fixture deliberately sends synthetic spans for workflow assertions; the separate Java script proves actual agent telemetry. Do not describe one as the other.

Remaining work

Reviewed mapping overrides, broader proxy/framework adapters, inherited/composed route resolution, application-server/WAR runtime certification, remote collector association, scenario tracking, projected latency distributions, independent JaCoCo coverage and production overhead/scale measurement are not implemented by this foundation slice. The existing runtime heuristic scores are unchanged and uncalibrated. Phase 4 canonical evidence/facts/twin and later modernization execution remain separate roadmap work.

OpenTelemetry references: Java configuration, method instrumentation, OTLP protocol.